Windows x86

Buffer Overflow PoC

for Beginners

A simple, self-contained tutorial on stack-based buffer overflow exploits for 32-bit Windows (educational only)

https://github.com/nataliadiak/windows-x86-shellcode-poc

Nataliia Dyak — April 2026

▸ No external tools required — everything you need is here.

What is this repository?

vulnerable.c

A simple program with an unsafe gets() function that reads user input without any bounds checking

messagebox.asm

Small assembly payload that executes after the overflow — displays a Windows MessageBox

exploit.c

Example of how the overflow payload is carefully constructed byte-by-byte

KEY TAKEAWAY

This project teaches one core idea: unsafe input reading can let an attacker overwrite the return address and redirect execution.

The Vulnerability — gets() has no size limit

```
char buffer[32];
gets(buffer);
```

WHAT THE PROGRAM DOES

The program reserves 32 bytes for buffer on the stack. This is the intended storage space for user input.

WHAT GETS() DOES

gets() does not check the buffer size. It keeps reading characters until it encounters a newline — no matter how many bytes that is.

CONSEQUENCE

If the user types 40 or 50 characters, the extra characters overflow past the 32-byte buffer — corrupting critical stack data.

How memory is laid out on the stack

Lower addresses (top of stack)

buffer (32 bytes)

saved EBP (4 bytes)

return address (4 bytes)

Higher addresses (bottom)

⬇ OVERFLOW DIRECTION ⬇

1. **Local Variables**
   Like our buffer — where user input is stored

2. **Saved EBP**
   Base pointer from the calling function

3. **Return Address**
   Where the CPU jumps when the function returns — the attacker's target

How the exploit works — Step by step

1. The program starts and allocates buffer[32] on the stack
2. It calls gets(buffer) to read a line of user input
3. We send a string longer than 32 bytes (say, 50 bytes)
4. gets() has no size check, so it writes all 50 bytes
5. The extra 18 bytes overflow past buffer and overwrite the saved EBP and return address
6. We carefully construct the overflow so the return address points to shellcode on the stack
7. When the function returns, the CPU jumps to the overwritten return address
8. The shellcode executes — in our case, displaying a MessageBox
9. The shellcode runs under the program's permissions

> This is the simplest form of code execution through a buffer overflow.

What happens inside messagebox.asm

1

LoadLibraryA

Loads "USER32.DLL" into memory so MessageBoxA is available

→

2

Prepare Args

Push 4 arguments onto the stack: handle, message, caption, type

→

3

MessageBoxA

Calls the Windows API — displays "CAN I HACK THE PC?"

→

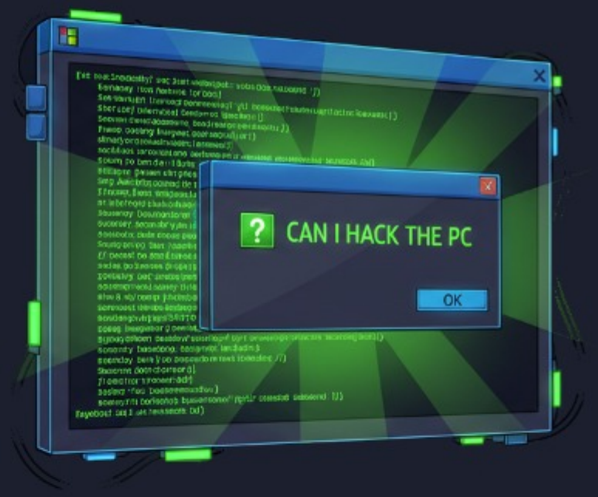
4

ExitProcess

Cleanly terminates the program to avoid a crash

Each step uses the x86 calling convention — arguments pushed right-to-left onto the stack

Live Demo



1. Buffer overflow triggered
2. Return address overwritten
3. Shellcode executed
4. MessageBox pops up

```
"CAN I HACK THE PC?"
```

How to build and run the demo

1 Disable Protections

- `/GS-` Stack Canaries
- `/DYNAMICBASE:NO` ASLR
- `/NXCOMPAT:NO` DEP/NX

2 Compile on Windows

```
cl /c /GS- /Z7 vulnerable.c
link /SUBSYSTEM:CONSOLE /DYNAMICBASE:NO /NXCOMPAT:NO
vulnerable.obj /OUT:vulnerable.exe
```

3 Run It

- Execute vulnerable.exe
- Provide long input that overflows the buffer
- Watch the shellcode execute
- **MessageBox appears!**

Already have the .exe? Disable protections with: `editbin /NXCOMPAT:NO /DYNAMICBASE:NO vulnerable.exe`

Why the README is complete

This README explains:

- What gets() is and why it is unsafe
- How the stack stores local variables, saved registers, and return addresses
- How an overflow can overwrite the return address
- How the CPU uses the return address when a function returns
- How shellcode can execute when the return address points to it
- What protections exist and why we disable them
- How to build and run the example

Important Safety Notice

This example is for learning only.

Do not use this technique against systems you do not own or have explicit permission to test.

Unauthorized access to computer systems is illegal.

Always practice security research responsibly and ethically.

Try it yourself

Clone the repo

Get all three source files to your machine

Follow the build steps

Compile with protections disabled as shown

Experiment with payload lengths

Try different input sizes and observe what happens

Compare the code files

Read each file alongside this explanation

```
github.com/nataliadiak/windows-x86-shellcode-poc
```

Thank you!

Thank you for checking out this educational PoC!

Star the repo
if it helped you understand buffer overflows

Questions or suggestions?
Open an issue on GitHub

@nataliadiak